Vishal Acharya

pd.concat() in Pandas — Detailed Explanation

pd.concat() is used to combine (concatenate) multiple Pandas objects such as DataFrames or Series along rows or columns.

Syntax: pd.concat(objs, axis=0, join='outer', ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, sort=False, copy=True)

Parameter	Purpose
`objs`	Objects to concatenate
`axis`	Rows (0) or Columns (1)
`join`	Outer / Inner alignment
`ignore_index`	Reset index
`keys`	Create MultiIndex
`levels`	Define MultiIndex levels
`names`	Name MultiIndex
`verify_integrity`	Check duplicate index
`sort`	Sort labels
`copy`	Copy data or not

objs

List or dictionary of Series/DataFrames to concatenate.

list → [df1, df2]

tuple → (df1, df2)

dict → {'A': df1, 'B': df2}

In [1]:
```
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})

result = pd.concat([df1, df2])
print(result)
```

```
   A  B
0  1  3
1  2  4
0  5  7
1  6  8
```

axis (Default = 0)

Determines direction of concatenation

Value Meaning

0 Row-wise (vertical)

1 Column-wise (horizontal)

In [4]:
```
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})

Row = pd.concat([df1, df2],axis=0)
print("RoW","\n",Row)
```

Vishal Acharya

Vishal Acharya

```
RoW
    A  B
0  1  3
1  2  4
0  5  7
1  6  8
```

In [5]:
```python
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})

col = pd.concat([df1, df2],axis=1)
print("Col","\n",col)
```

```
Col
    A  B  A  B
0  1  3  5  7
1  2  4  6  8
```

join (Default = 'outer')

Controls how indexes/columns are aligned

Value	Meaning
`'outer'`	Union (keeps all labels)
`'inner'`	Intersection (common labels only)

inner

In [10]:
```python
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "d":[7,8]})

result = pd.concat([df1, df2], join='inner')
print(result)
```

```
   A
0  1
1  2
0  5
1  6
```

Outer

In [11]:
```python
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "d":[7,8]})

result = pd.concat([df1, df2], join='outer')
print(result)
```

```
   A    B    d
0  1  3.0  NaN
1  2  4.0  NaN
0  5  NaN  7.0
1  6  NaN  8.0
```

ignore_index (Default = False)

If True, reset index

Old index is discarded

Useful when row numbers don't matter

In [12]:
```python
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})

result = pd.concat([df1, df2], ignore_index=True)
print(result)
```

Vishal Acharya

Vishal Acharya

```
   A  B
0  1  3
1  2  4
2  5  7
3  6  8
```

In [13]:
```python
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})

result = pd.concat([df1, df2], ignore_index=False)
print(result)
```

```
   A  B
0  1  3
1  2  4
0  5  7
1  6  8
```

Keys

Adds hierarchical (MultiIndex) labels

In [14]:
```python
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})

result = pd.concat([df1, df2], keys=['Train', 'Test'])
print(result)
```

```
        A  B
Train 0 1  3
      1 2  4
Test  0 5  7
      1 6  8
```

levels

Defines custom levels for MultiIndex (rarely used directly)

In [23]:
```
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})

result =pd.concat([df1, df2], keys=['A', 'B'], levels=[['A', 'B']])

print(result)
```

```
     A  B
A 0  1  3
  1  2  4
B 0  5  7
  1  6  8
```

Names the MultiIndex levels

In [24]:
```
pd.concat([df1, df2], keys=['A', 'B'], names=['Dataset'])
```

Out[24]:

		A	B
Dataset			
A	**0**	1	3
	1	2	4
B	**0**	5	7
	1	6	8

verify_integrity (Default = False)

If True, checks for duplicate index values

Raises error if duplicates exist

Vishal Acharya

Error if overlapping index found

In [25]:
```
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})
pd.concat([df1, df2], verify_integrity=True)
```

Vishal Acharya

Vishal Acharya

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-25-1b021d53381d> in <module>
      3 df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
      4 df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})
----> 5 pd.concat([df1, df2], verify_integrity=True)

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\reshape\concat.py in concat(objs, axis, join, ignore_index, keys, level
s, names, verify_integrity, sort, copy)
    272     ValueError: Indexes have overlapping values: ['a']
    273     """
--> 274     op = _Concatenator(
    275         objs,
    276         axis=axis,

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\reshape\concat.py in __init__(self, objs, axis, join, keys, levels, name
s, ignore_index, verify_integrity, copy, sort)
    452         self.copy = copy
    453
--> 454         self.new_axes = self._get_new_axes()
    455
    456     def get_result(self):

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\reshape\concat.py in _get_new_axes(self)
    517     def _get_new_axes(self) -> List[Index]:
    518         ndim = self._get_result_dim()
--> 519         return [
    520             self._get_concat_axis() if i == self.bm_axis else self._get_comb_axis(i)
    521             for i in range(ndim)

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\reshape\concat.py in <listcomp>(.0)
    518         ndim = self._get_result_dim()
    519         return [
--> 520             self._get_concat_axis() if i == self.bm_axis else self._get_comb_axis(i)
    521             for i in range(ndim)
    522         ]

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\reshape\concat.py in _get_concat_axis(self)
    578             )
    579
```

Vishal Acharya

```
--> 580         self._maybe_check_integrity(concat_axis)
    581
    582         return concat_axis

C:\ProgramData\Anaconda3\lib\site-packages\pandas\core\reshape\concat.py in _maybe_check_integrity(self, concat_index)
    586             if not concat_index.is_unique:
    587                 overlap = concat_index[concat_index.duplicated()].unique()
--> 588                 raise ValueError(f"Indexes have overlapping values: {overlap}")
    589
    590

ValueError: Indexes have overlapping values: Int64Index([0, 1], dtype='int64')
```

In [26]:
```
import pandas as pd

df1 = pd.DataFrame({"A":[1,2], "B":[3,4]})
df2 = pd.DataFrame({"A":[5,6], "B":[7,8]})
pd.concat([df1, df2], verify_integrity=False)
```

Out[26]:

	A	B
0	1	3
1	2	4
0	5	7
1	6	8

sort (Default = False)

Sorts columns/index if they don't align

Useful when column order matters

In [33]:
```
import pandas as pd

df1 = pd.DataFrame({"E":[9,10], "B":[3,4]})
```

```
df2 = pd.DataFrame({"C":[5,6], "D":[7,8]})
pd.concat([df1, df2], sort=True)
```

Out[33]:

	B	C	D	E
0	3.0	NaN	NaN	9.0
1	4.0	NaN	NaN	10.0
0	NaN	5.0	7.0	NaN
1	NaN	6.0	8.0	NaN

In [34]:
```
pd.concat([df1, df2], sort=False)
```

Out[34]:

	E	B	C	D
0	9.0	3.0	NaN	NaN
1	10.0	4.0	NaN	NaN
0	NaN	NaN	5.0	7.0
1	NaN	NaN	6.0	8.0

copy (Default = True)

If True, copies data

If False, avoids copying (memory-efficient)

In [35]:
```
pd.concat([df1, df2], copy=False)
```

Vishal Acharya

Vishal Acharya

Out[35]:

	E	B	C	D
0	9.0	3.0	NaN	NaN
1	10.0	4.0	NaN	NaN
0	NaN	NaN	5.0	7.0
1	NaN	NaN	6.0	8.0

In [36]:
```
pd.concat([df1, df2], copy=True)
```

Out[36]:

	E	B	C	D
0	9.0	3.0	NaN	NaN
1	10.0	4.0	NaN	NaN
0	NaN	NaN	5.0	7.0
1	NaN	NaN	6.0	8.0

Feature	concat	append
Multiple objects	✅	❌
Performance	Fast	Slow
Recommended	✅	❌ (deprecated)

MultiIndex → Fetch using iloc

Create MultiIndex DataFrame

iloc ignores index labels → works only on position

In [37]:
```
arrays = [['A','A','B','B'], [1,2,1,2]]
index = pd.MultiIndex.from_arrays(arrays)
```

```
df = pd.DataFrame({"Marks":[70,80,85,90]}, index=index)
print(df)
```

```
     Marks
A 1     70
  2     80
B 1     85
  2     90
```

In [38]:
```
df.iloc[2]
```

Out[38]:
```
Marks    85
Name: (B, 1), dtype: int64
```

pd.merge() in Pandas (ALL SQL Joins)

pd.merge() is used to combine two DataFrames based on common columns or indexes, similar to SQL JOIN operations.

Syntax: pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None, left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'), copy=True, indicator=False, validate=None)

SQL JOIN	Pandas `how`
INNER JOIN	`how='inner'`
LEFT JOIN	`how='left'`
RIGHT JOIN	`how='right'`
FULL OUTER JOIN	`how='outer'`
CROSS JOIN	`how='cross'`

Parameter	Purpose
`how`	Type of SQL JOIN
`on`	Common column

Vishal Acharya

Vishal Acharya

Parameter	Purpose
`left_on/right_on`	Different column names
`left_index/right_index`	Merge on index
`suffixes`	Resolve duplicate columns
`indicator`	Track row source
`validate`	Enforce merge rules

- Employees DataFrame

Each employee belongs to a department

Department 30 exists only in employees

- Departments DataFrame

Department 40 exists only in departments

No employee in dept 40

INNER JOIN — how='inner'

- What happens?

Keeps only rows where dept_id exists in BOTH tables

- Matching dept_id

10 → present in both

20 → present in both

30 → only in employees ❌

40 → only in departments ❌

emp	dept
1,3	HR
2	IT

Most common join

Equivalent to SQL INNER JOIN

In [39]:
```
import pandas as pd

employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['A', 'B', 'C', 'D'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20, 40],
    'dept_name': ['HR', 'IT', 'Finance']
})

pd.merge(employees, departments, on='dept_id', how='inner')
```

Out[39]:

	emp_id	name	dept_id	dept_name
0	1	A	10	HR
1	3	C	10	HR
2	2	B	20	IT

LEFT JOIN — how='left'

- What happens?

Vishal Acharya

Vishal Acharya

Keeps ALL rows from left table (employees)

Adds matching data from departments

If no match → NaN

- Key Point

Employee D (dept 30) has no department → NaN

- Use Case

When left table is primary

Example: employee list must never shrink

In [40]:
```
import pandas as pd

employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['A', 'B', 'C', 'D'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20, 40],
    'dept_name': ['HR', 'IT', 'Finance']
})
pd.merge(employees, departments, on='dept_id', how='left')
```

Vishal Acharya

Out[40]:

	emp_id	name	dept_id	dept_name
0	1	A	10	HR
1	2	B	20	IT
2	3	C	10	HR
3	4	D	30	NaN

RIGHT JOIN — how='right'

- What happens?

Keeps ALL rows from right table (departments)

Adds employee data if available

Missing employee → NaN

- Key Point

Dept 40 (Finance) has no employees

- Use Case

When right table is master data

In [41]:

```
import pandas as pd

employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['A', 'B', 'C', 'D'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20, 40],
```

Vishal Acharya

```
    'dept_name': ['HR', 'IT', 'Finance']
})
pd.merge(employees, departments, on='dept_id', how='right')
```

Out[41]:

	emp_id	name	dept_id	dept_name
0	1.0	A	10	HR
1	3.0	C	10	HR
2	2.0	B	20	IT
3	NaN	NaN	40	Finance

FULL OUTER JOIN — how='outer'

- What happens?

Keeps ALL rows from BOTH tables

Missing matches filled with NaN

- Result Includes

Employee D (dept 30)

Department Finance (dept 40)

- Use Case

Data auditing

Finding mismatches between tables

In [42]:

```
import pandas as pd

employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
```

Vishal Acharya

```
    'name': ['A', 'B', 'C', 'D'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20, 40],
    'dept_name': ['HR', 'IT', 'Finance']
})

pd.merge(employees, departments, on='dept_id', how='outer')
```

Out[42]:

	emp_id	name	dept_id	dept_name
0	1.0	A	10	HR
1	3.0	C	10	HR
2	2.0	B	20	IT
3	4.0	D	30	NaN
4	NaN	NaN	40	Finance

CROSS JOIN — how='cross'

- What happens?

Every employee × every department

Cartesian Product

- Rows = 4 × 3 = 12
- Use Case

1. Combinations
2. Simulations
3. Recommendation systems

- No on, left_on, or right_on allowed

Vishal Acharya

In [1]:
```python
import pandas as pd

employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['A', 'B', 'C', 'D'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20, 40],
    'dept_name': ['HR', 'IT', 'Finance']
})

# Correct Cross Join: No 'on' or 'left_on/right_on' parameters
result = pd.merge(employees, departments, how='cross')

print(result)
```

```
    emp_id name  dept_id_x  dept_id_y dept_name
0        1    A         10         10        HR
1        1    A         10         20        IT
2        1    A         10         40   Finance
3        2    B         20         10        HR
4        2    B         20         20        IT
5        2    B         20         40   Finance
6        3    C         10         10        HR
7        3    C         10         20        IT
8        3    C         10         40   Finance
9        4    D         30         10        HR
10       4    D         30         20        IT
11       4    D         30         40   Finance
```

Different Column Names

left_on, right_on

- Why needed?

Column names differ:

Vishal Acharya

dept_id ≠ d_id

- What happens?

Pandas matches values, not names

Both columns remain unless dropped

- Real-World Case

Joining CSV files from different sources

In [7]:
```
import pandas as pd

employees = pd.DataFrame({
    'emp_id': [1, 2],
    'name': ['A', 'B'],
    'dept_id': [10, 20]
})

dept_alt = pd.DataFrame({
    'd_id': [10, 20],
    'dept_name': ['HR', 'IT']
})

# Performing the merge
result = pd.merge(employees, dept_alt, left_on='dept_id', right_on='d_id')

print(result)
```

```
   emp_id name  dept_id  d_id dept_name
0       1    A       10    10        HR
1       2    B       20    20        IT
```

Cleaning up the Duplicate

In a production script, you usually don't want both dept_id and d_id because they contain identical information. You can "chain" a drop command to clean it up immediately:

Parameter:When to use it

on: When column names are identical in both DataFrames.

left_on / right_on: When column names are different but the data matches.

left_index / right_index: When you want to join using the row labels (index) instead of a column.

In [8]:
```
result = pd.merge(
    employees,
    dept_alt,
    left_on='dept_id',
    right_on='d_id'
).drop('d_id', axis=1) # Removes the redundant column
result
```

Out[8]:

	emp_id	name	dept_id	dept_name
0	1	A	10	HR
1	2	B	20	IT

Vishal Acharya

Vishal Acharya

Merge Using Index

left_index, right_index

- What happens?

Merge happens on index values

No column key needed

- When to use?

1. After set_index()
2. Faster joins
3. Cleaner code

In [11]:
```
import pandas as pd

employees = pd.DataFrame({
    'emp_id': [1, 2],
    'name': ['A', 'B'],
    'dept_id': [10, 20]
})

dept_alt = pd.DataFrame({
    'd_id': [10, 20],
    'dept_name': ['HR', 'IT']
})

pd.merge(
     employees,
    dept_alt,
    left_index=True,
    right_index=True
)
```

Out[11]:

	emp_id	name	dept_id	d_id	dept_name
0	1	A	10	10	HR
1	2	B	20	20	IT

Duplicate Column Names suffixes

- Problem

Both tables have column dept_name

- Solution

Pandas renames automatically using suffixes

- Without suffix?

Confusing column overwrite

- What happens "Without Suffixes"?

If you are asking what happens if you try to disable this behavior, there are a few scenarios:

Logic Error: You cannot have two columns with the exact same name in a single DataFrame. Pandas will force a rename regardless.

Manual Overwrite: If you manually rename columns after the merge to be identical, many pandas operations will fail or behave unpredictably because the "labels" are no longer unique.

Confusing Data: Without clear suffixes, you might accidentally use the updated_at from the department table when you actually needed the employee record's timestamp.

By default, pandas appends _x to the column from the left table and _y to the column from the right table.

In [1]:
```
import pandas as pd
```

Vishal Acharya

Vishal Acharya

```
employees = pd.DataFrame({
    'emp_id': [1, 2],
    'dept_id': [10, 20],
    'updated_at': ['2023-01-01', '2023-01-01'] # Shared name
})

departments = pd.DataFrame({
    'dept_id': [10, 20],
    'dept_name': ['HR', 'IT'],
    'updated_at': ['2023-05-01', '2023-05-01'] # Shared name
})

# Default merge
result = pd.merge(employees, departments, on='dept_id')
print(result.columns)
# Output: Index(['emp_id', 'dept_id', 'updated_at_x', 'dept_name', 'updated_at_y'], dtype='object')
result
```

```
Index(['emp_id', 'dept_id', 'updated_at_x', 'dept_name', 'updated_at_y'], dtype='object')
```

Out[1]:

	emp_id	dept_id	updated_at_x	dept_name	updated_at_y
0	1	10	2023-01-01	HR	2023-05-01
1	2	20	2023-01-01	IT	2023-05-01

Customizing Suffixes

The default _x and _y can be cryptic. You can provide a suffixes tuple to make the data much clearer (e.g., source names or timestamps).

In [16]:

```
result = pd.merge(
    employees,
    departments,
    on='dept_id',
    suffixes=('_emp', '_dept')
)
result
```

Out[16]:

	emp_id	dept_id	updated_at_emp	dept_name	updated_at_dept
0	1	10	2023-01-01	HR	2023-05-01
1	2	20	2023-01-01	IT	2023-05-01

In [1]:
```
import numpy as np
import pandas as pd
```

file name -> Columns

quater-i.csv -> ['order_id', 'quantity', 'item_id', 'choice_description_id' 'item_price']

items.csv -> ['item_id', 'item_name']

In [2]:
```
# import like this
items_path = "items.csv"
q1_path = "quarter-1.csv"
q2_path = "quarter-2.csv"
q3_path = "quarter-3.csv"


q1= pd.read_csv(q1_path)
q2 = pd.read_csv(q2_path)
q3 = pd.read_csv(q3_path)

items = pd.read_csv(items_path)
```

`Q:1-5`

1. You are given three quater files, your job is to append these three files and make a single dataframe.
2. Have a index as Q-1 Q-2 Q-3 for respective quater files in the dataframe
3. Your are given a file items.csv which has item_id and item_name. Find out most sold items in each quarter.
4. Find out items which has made most revenue in each quarter.
5. Find out avg order price of each quarter.

Vishal Acharya

Vishal Acharya

Note: item_price is given as str with $ sign, in earlier task you have converted this to rupees, here too first convert item_price field in rupees.

In [3]:
```
df1 = pd.concat([q1, q2, q3], keys=["Q1", "Q2", "Q3"])
df1
```

Out[3]:

		order_id	quantity	item_id	choice_description_id	item_price
Q1	**0**	1	1	1	1	$3.39
	1	1	1	2	2	$3.39
	2	2	2	4	3	$16.98
	3	4	1	7	6	$9.25
	4	6	1	9	8	$8.75
...	**...**	...	...	...	...	...
Q2	**2342**	1829	1	23	92	$11.25
	2343	1830	1	23	1043	$11.25
	2344	1832	1	10	116	$8.75
	2345	1832	1	8	0	$4.45
	2346	1834	1	20	515	$11.25

4622 rows × 5 columns

In [4]:
```
new_df = df1.reset_index().merge(items, on="item_id")
new_df.rename(columns={"level_0": "quarter"}, inplace=True)
new_df.groupby(["quarter", "item_name"], as_index = False)["quantity"].sum().sort_values("quantity", ascending = False).drop_d
```

Vishal Acharya

Out[4]:

	quarter	item_name	quantity
65	Q2	Chicken Bowl	394
17	Q1	Chicken Bowl	367

In [5]:

```
#4
new_df["item_price"] = new_df.item_price.apply(lambda x: float(x[1:]))
new_df["total_item_price"] = new_df["item_price"] * new_df["quantity"]
new_df.groupby(["quarter", "item_name"], as_index = False)["total_item_price"].sum().sort_values("total_item_price", ascending
```

Out[5]:

	quarter	item_name	total_item_price
65	Q2	Chicken Bowl	4192.25
17	Q1	Chicken Bowl	3852.38

In [6]:

```
#5
new_df.groupby(["quarter", "order_id"], as_index = False)["total_item_price"].sum().groupby("quarter", as_index = False)["tota
```

Out[6]:

	quarter	total_item_price
0	Q1	13.809488
1	Q2	13.279828

`Q-6` From the IPL wala dataset you have to find the Purple cap holder each season.

Note: Bowler with most no wickets in a season gets purple cap. If more than one bowler have same no of wickets in the season, one with least ecomnomy among them is purple cap holder.

Bowler's Economy = runs-conceded per six ballsm

In [8]:

```
balls = pd.read_csv("IPL_Ball_by_Ball_2008_2022.csv")
matches = pd.read_csv("IPL_Matches_2008_2022.csv")
```

Vishal Acharya

In [9]:
```
seasondf = balls.merge(matches[["ID", "Season"]], on="ID")
seasondf["IsBowlerWicket"] = seasondf.kind.apply(lambda x: 1 if x in ["caught", 'caught and bowled', 'bowled', 'stumped','lbw'
seasondf["BowlerRun"] = seasondf.extra_type.apply(lambda x: 0 if x in ["legbyes", "byes"] else 1) * seasondf["total_run"]
seasondf["IsLegalBall"] = seasondf.extra_type.apply(lambda x: 0 if x in ["wides", "noballs"] else 1)
pcapdf = seasondf.groupby(["Season", "bowler"], as_index = False)[["IsBowlerWicket", "BowlerRun", "IsLegalBall"]].sum()
pcapdf["Economy"] = pcapdf["BowlerRun"] / pcapdf["IsLegalBall"] * 6
pcapdf.sort_values(["IsBowlerWicket", "Economy"], ascending = [False, True]).drop_duplicates("Season", keep = "first").sort_va
```

Out[9]:

	Season	bowler	IsBowlerWicket	BowlerRun	IsLegalBall	Economy
84	2007/08	Sohail Tanvir	22	266	247	6.461538
174	2009	RP Singh	23	417	358	6.988827
284	2009/10	PP Ojha	21	429	353	7.291785
447	2011	SL Malinga	28	375	378	5.952381
537	2012	M Morkel	25	453	378	7.190476
629	2013	DJ Bravo	32	497	375	7.952000
779	2014	MM Sharma	23	452	323	8.396285
847	2015	DJ Bravo	26	426	314	8.140127
938	2016	B Kumar	23	490	396	7.424242
1048	2017	B Kumar	26	369	314	7.050955
1144	2018	AJ Tye	24	448	336	8.000000
1269	2019	Imran Tahir	26	431	386	6.699482
1381	2020/21	K Rabada	32	549	398	8.276382
1467	2021	HV Patel	32	459	338	8.147929
1671	2022	YS Chahal	27	527	408	7.750000

Best bowler in death overs. Note: Have taken most no of wickets in case of tie with least economy

Death Overs - [16-20]

In [10]:
```
death_overs = seasondf[seasondf.overs>=15]
pcapdf = death_overs.groupby("bowler", as_index = False)[["IsBowlerWicket", "BowlerRun", "IsLegalBall"]].sum()
pcapdf["Economy"] = pcapdf["BowlerRun"] / pcapdf["IsLegalBall"] * 6
pcapdf.sort_values(["IsBowlerWicket", "Economy"], ascending = [False, True]).head()
```

Out[10]:

	bowler	IsBowlerWicket	BowlerRun	IsLegalBall	Economy
91	DJ Bravo	115	2161	1385	9.361733
331	SL Malinga	108	1464	1117	7.863921
53	B Kumar	90	1752	1173	8.961637
148	JJ Bumrah	80	1614	1146	8.450262
339	SP Narine	67	1164	949	7.359326

`Q-8` Batsman record season wise

Make a function which takes a input `batsman_name` and it returns a dataframe. Columns of the data frame are - `['Season','Innings', 'TotalRuns', 'Avg', 'HighestScore','StrikeRate']`.

- In result make `Season` column as index.
- Avg - total_runs/ no of time got out. - player_out column will help.
- StrikeRate -(total_runs/ balls faced) * 100- wides are not included in batsman ball faced counts. No balls are included. -> Extra_type column will help
- Batsman Can score runs on No Balls.
- Batsman can get out on No Ball or Wides. And even while being on non-striker. Keep these things in mind before masking.

In [11]:
```
# code here
batterdf = seasondf.copy()
```

Vishal Acharya

```python
batterdf["IsBatsmanBall"] = batterdf["extra_type"].apply(lambda x: 1 if x != "wides" else 0)

def bat_record_season(batsman):
    bdf = batterdf[batterdf.batter == batsman].copy()
    bdf["IsBatsmanOut"] = bdf.batter == bdf.player_out
    df = bdf.groupby(["Season", "ID"], as_index = False)[["batsman_run", "IsBatsmanBall", "IsBatsmanOut"]].sum()
    innings = df.groupby("Season").ID.count()
    df = df.groupby("Season").agg(
    {
        "batsman_run":["sum", "max"],
        "IsBatsmanBall": "sum",
        "IsBatsmanOut": "sum"
    })
    df["Innings"]= innings
    df["TotalRuns"] = df[("batsman_run", "sum")]
    df["Avg"] = df["TotalRuns"] / df[("IsBatsmanOut", "sum")]
    df["HighestScore"] = df[("batsman_run", "max")]
    df["StrikeRate"] = df["TotalRuns"] / df[("IsBatsmanBall", "sum")] * 100
    return df.drop(columns = ["batsman_run", "IsBatsmanBall", "IsBatsmanOut"])
bat_record_season("MS Dhoni")
```

Vishal Acharya

Vishal Acharya

Out[11]:

Season	Innings	TotalRuns	Avg	HighestScore	StrikeRate
2007/08	14	414	41.400000	65	133.548387
2009	13	332	41.500000	58	127.203065
2009/10	11	287	31.888889	66	136.666667
2011	13	392	43.555556	70	158.704453
2012	17	358	32.545455	51	128.776978
2013	16	461	46.100000	67	162.897527
2014	15	371	74.200000	57	148.400000
2015	17	372	31.000000	53	121.967213
2016	12	284	40.571429	64	135.238095
2017	15	290	26.363636	61	116.000000
2018	15	455	75.833333	79	150.662252
2019	12	416	104.000000	84	134.627832
2020/21	12	200	25.000000	47	116.279070
2021	11	114	16.285714	18	106.542056
2022	12	232	33.142857	50	123.404255

`Q-9` Using both dataset, make a dataframe as described below

Data Frame columns-> `['PlayerOfThematch', 'BattingFigure', 'BowlingFigure']`

- BattingFigure-> `<runs>/<balls>`
- BowlingFigure-> `<wicket>/<runs-conceded>`

Vishal Acharya

DataFrame should have one record for each match.

Say 'V Kohli' got POM award then in dataset include his batting figure of that match. Say he scored 112runs in 76 balls. And he hasn't bowled so Bowling Figure will be NaN

```
PlayerOfThematch BattingFigure BowlingFigure
V Kohli          112/76         nan
```

In [12]:
```
# code here
df = balls.merge(matches[["ID", "Player_of_Match"]], on="ID")
batterdf = df[df.batter == df.Player_of_Match].copy()
batterdf["IsBatsmanBall"] = batterdf["extra_type"].apply(lambda x: 1 if x != "wides" else 0)
batter = batterdf.groupby(["ID", "batter"], as_index=False)[["batsman_run", "IsBatsmanBall"]].sum()
batter["BattingFigure"] = batter[["batsman_run", "IsBatsmanBall"]].apply(lambda x: '/'.join(map(str, x.values)), axis = 1)
batter.rename(columns = {"batter": "PlayerOfMatch"}, inplace = True)
batter.head()
```

Out[12]:

	ID	PlayerOfMatch	batsman_run	IsBatsmanBall	BattingFigure
0	335982	BB McCullum	158	73	158/73
1	335983	MEK Hussey	116	54	116/54
2	335985	MV Boucher	39	19	39/19
3	335986	DJ Hussey	38	43	38/43
4	335987	SR Watson	76	49	76/49

In [13]:
```
bowlerdf = df[df.bowler == df.Player_of_Match].copy()
bowlerdf["IsBowlerWicket"] = bowlerdf["kind"].apply(lambda x: 1 if x in ["caught", 'caught and bowled', 'bowled', 'stumped','l
bowlerdf["BowlerRun"] = bowlerdf.extra_type.apply(lambda x: 0 if x in ["legbyes", "byes"] else 1) * bowlerdf["total_run"]
bowler = bowlerdf.groupby(["ID", "bowler"], as_index=False)[["IsBowlerWicket", "BowlerRun"]].sum()
bowler["BowlingFigure"] = bowler[["IsBowlerWicket", "BowlerRun"]].apply(lambda x: '/'.join(map(str, x.values)), axis = 1)
bowler.rename(columns = {"bowler": "PlayerOfMatch"}, inplace = True)
bowler.head()
```

Out[13]:

	ID	PlayerOfMatch	IsBowlerWicket	BowlerRun	BowlingFigure
0	335984	MF Maharoof	2	11	2/11
1	335986	DJ Hussey	1	35	1/35
2	335987	SR Watson	0	39	0/39
3	335990	YK Pathan	2	20	2/20
4	335992	SR Watson	2	20	2/20

In [14]:

```
batter.merge(bowler, on=["ID", "PlayerOfMatch"], how = "outer").drop(columns = ["batsman_run", "IsBatsmanBall", "IsBowlerWicke
```

Out[14]:

	ID	PlayerOfMatch	BattingFigure	BowlingFigure
0	335982	BB McCullum	158/73	NaN
1	335983	MEK Hussey	116/54	NaN
2	335984	MF Maharoof	NaN	2/11
3	335985	MV Boucher	39/19	NaN
4	335986	DJ Hussey	38/43	1/35

In [ ]:

Vishal Acharya